

COMP 6721 APPLIED ARTIFICIAL INTELLIGENCE

FACE DETECTION AND RECOGNITION APP

PROJECT BY

DISHA PADSALA (40324578)

YASH DILIPBHAI KOLADIYA (40324106)

MANUSH PARAGKUMAR KHATRI (40329710)



INDEX

1. Introduction	1
2. Dataset Collection	1
3. Data Preprocessing	3
4. Model Architecture.....	5
5. Training Details	7
6. Evaluation Results	9
7. Threshold Selection	11
8. System Implementation.....	11
9. Limitations.....	12
10. Future Improvements	12
11. Conclusion and References	13

Face Recognition System Using Convolutional Neural Networks

1. Introduction

This project presents the design and implementation of a real-time face recognition system using a custom Convolutional Neural Network (CNN) trained on a self-collected dataset of 1577 face images across 8 individuals. The system detects faces from a live webcam feed, preprocesses them, and classifies them against a set of known identities. If the model's confidence exceeds a threshold of 0.7, the person is identified by name and logged to an attendance record; otherwise they are labeled as "Unknown." A key finding of this project was that data quality — particularly lighting consistency — had a far greater impact on accuracy than model complexity, with overall accuracy improving from 64% to 97% after re-recording videos under better conditions.

2. Dataset Collection

2.1 Overview

We collected our own dataset rather than using a pre-existing one, as the primary goal was to recognize specific individuals in real-world webcam conditions. The dataset includes 3 project team members — Disha, Manush, and Yash — along with 5 additional volunteers: Rahil, Shravani, Sidhhi, Soham, and Tisha, who are friends and classmates. All participants gave their informed consent to be photographed and included in the dataset.

2.2 Image Requirements

The project required a minimum of 30 images per person with variety across poses, lighting conditions, facial expressions, angles, and styles. We significantly exceeded this minimum, collecting approximately 186–208 images per person for a total of 1577 images. To satisfy the variety requirements, each participant recorded a 30-second video following these guidelines:

- **Poses:** Natural head movement including looking straight, left, right, up, and down
- **Lighting conditions:** Bright indoor lighting near a window or lamp; some participants recorded in multiple rooms to introduce lighting variation
- **Facial expressions:** Neutral, smiling, talking, and frowning expressions
- **Angles:** Frontal and non-frontal angles captured naturally through head movement
- **Styles:** Participants recorded with and without glasses where applicable

2.3 Initial Approach — Celebrity Data

In our initial approach, we attempted to supplement our dataset with celebrity images downloaded from the internet using the Bing Image Search crawler. The intent was to increase the number of identities in the dataset and make the model more robust. However, this approach introduced a significant domain mismatch problem. Internet images of celebrities vary widely in resolution, lighting, background, camera angle, compression artifacts, and image style — professional photoshoots, paparazzi photos, red carpet events, and movie stills all look fundamentally different from smartphone-recorded face crops. When mixed with our phone-recorded images, the model struggled to learn consistent facial features and produced poor accuracy across all classes. After extensive experimentation we concluded that training data must match the deployment conditions as closely as possible, and we abandoned the celebrity dataset entirely in favor of a fully self-collected dataset recorded under consistent conditions.

2.4 Recording Conditions and Iteration

Early experiments with our self-collected dataset also revealed that lighting quality had a dramatic effect on model performance. Our first round of videos was recorded in dim lighting with inconsistent angles, producing a model accuracy of only 64%. After re-recording all videos under the improved conditions described in Section 2.2, accuracy improved to 97%. This reinforced the lesson learned from the celebrity data experiment — consistency and quality of recording conditions matter far more than the size of the dataset.

2.5 Face Extraction Pipeline

Face crops were extracted from each video using `gather_examples.py`, a script built around OpenCV's SSD deep learning face detector. The detector uses two model files — `deploy.prototxt` (network architecture) and `res10_300x300_ssd_iter_140000.caffemodel` (pretrained weights) — and processes each video as follows:

1. Each video frame is read sequentially
2. Every 4th frame is sampled (`--skip 4`) to avoid extracting near-duplicate images from consecutive frames
3. Each sampled frame is resized to 300×300 and passed through the SSD network as a blob with mean subtraction values (104.0, 177.0, 123.0)
4. The detection with the highest confidence score is selected
5. If the confidence exceeds 0.5, the face bounding box is extracted from the original frame
6. The cropped face is resized to 160×160 pixels and saved as a PNG file

The script is run from the command line for each person:

```
python3 gather_examples.py \
```

```
--input videos/Disha.mp4 \  
--output faces_dataset/Disha \  
--detector face_detector \  
--skip 4
```

2.6 Dataset Statistics

Person	Role	Images Collected
Disha	Team Member	208
Manush	Team Member	186
Yash	Team Member	200
Rahil	Volunteer	204
Shravani	Volunteer	193
Sidhhi	Volunteer	190
Soham	Volunteer	203
Tisha	Volunteer	193
Total		1577

All collected images exceed the minimum requirement of 30 per person by a factor of approximately 6 $\times$. The dataset is well balanced with approximately 186–208 images per person, ensuring no single class dominates training. All images are 160 $\times$ 160 pixels in color PNG format, stored in per-person subfolders under faces_dataset/.

3. Data Preprocessing

3.1 Image Loading and Grayscale Conversion

All images were loaded from faces_dataset/ directory with the help of OpenCV. As all the images had been pre-cropped to 160 $\times$ 160 pixel size using gather_examples.py file, therefore no extra work for face detection was required at this point. All images were resized to 160 $\times$ 160 pixels, and subsequently, the BGR to grayscale conversion was performed using cv2.COLOR_BGR2GRAY.

The process of converting an image to grayscale is intentional. There would be more variation based on illumination, different skin colours, and camera settings in a colour image, which adds

more complexity to input without any added benefit. Facial structures and details, which contribute to recognition, are retained in a grayscale image.

3.2 Train/Validation/Test Split

The dataset was split using stratified sampling with an 80/20 ratio:

- **Training set:** 1261 images (80%)
- **Validation/Test set:** 316 images (20%)

Stratified splitting (`stratify=all_labels`) ensures that each individual is equally represented within all splits, thus avoiding skewness in the test set, which would bias model evaluation. A fixed random seed (`random_state=42`) was chosen to allow for reproducible results.

A 20% portion of the data is set aside for use as the validation set during training and test set after the training process, when the classification report and confusion matrix are created. It is typical to reserve such a 20% portion when working with limited data because, otherwise, there will not be enough testing instances of each category to produce trustworthy results. The important condition here is to ensure that the set aside is never part of gradient descent – it only plays a role in validation.

The split data is saved into a Keras-compatible directory structure:

data/

train/

test/

3.3 Normalization

The rescale value is set at $1/255$, and all pixel values are normalized between 0 and 1. Normalizing the pixel values prevents gradient magnitude inconsistencies and helps achieve faster convergence.

3.4 Data Augmentation

Data augmentation was applied exclusively to the training set using Keras ImageDataGenerator to artificially increase diversity and reduce overfitting. The following augmentations were applied randomly to each training batch:

Augmentation	Value
--------------	-------

Rotation	$\pm 15^\circ$
----------	----------------

Zoom	$\pm 10\%$
------	------------

Width shift	$\pm 10\%$
-------------	------------

Augmentation Value

Height shift $\pm 10\%$

Horizontal flip Enabled

Brightness range [0.8, 1.2]

These transformations simulate real-world variation in head pose, distance, and lighting that the model will encounter during live webcam inference. Augmentation was not applied to the test set — only normalization — to ensure evaluation reflects true model performance on unseen data.

3.5 Data Generators

Three separate Keras ImageDataGenerator instances were used:

- **Training generator:** Applies full augmentation pipeline, loads images in batches of 20 with `color_mode='grayscale'`
- **Testing generator:** Applies normalization only, used during training for validation loss computation via `validation_data=testing_generator` in `model.fit()`
- **Validation generator:** Identical to testing generator but with `shuffle=False` to ensure consistent ordering for classification report and confusion matrix generation

4. Model Architecture

4.1 Design Decisions

A custom CNN had to be designed from scratch as part of this assignment. The path to creating the architecture entailed going through several stages. In our first attempts at creating a CNN, we employed small filters and fewer layers; the result was low accuracy because the model would predict the same class irrespective of the image fed into it. This happened because the model was too simple to capture the distinguishing features of 8 faces.

The process of tuning continued with changes to the architecture in which we added an additional layer of convolution blocks from two to three, increased the depth of the filters from 32 to 64 and further to 128, and applied L2 regularization (`weight=0.01`) to both convolutional and dense layers to avoid overfitting. Different activation functions were tried for the first convolution layer, and it was finally decided that a linear activation function would work better than ReLU to enable the network to learn the low-level edge features of the input grayscale image. The size of the dense layer was explored with 128, 256, and 512 neurons, out of which 512 yielded the best validation accuracy. Instead of Dropout, MCDropout was chosen since the latter provides uncertainty estimates of the prediction. The learning rate was reduced from 0.001 to 0.0003 to increase convergence stability. Patience in early stopping was increased to 10 iterations to provide enough time for convergence. All these steps led to validation accuracy rising from below 30% initially to 97%.

4.2 Architecture Overview

The model follows a classic CNN pattern — alternating convolutional and pooling layers for feature extraction, followed by fully connected layers for classification. Filter depth increases progressively (32 → 64 → 128) allowing the network to learn increasingly complex facial features at each stage.

Layer	Output Shape Parameters	
Conv2D (32 filters, 3×3, linear, same padding)	160×160×32	320
MaxPooling2D (2×2)	80×80×32	0
Conv2D (64 filters, 3×3, ReLU, L2=0.01)	78×78×64	18,496
MaxPooling2D (2×2)	39×39×64	0
Conv2D (128 filters, 3×3, ReLU, L2=0.01)	37×37×128	73,856
MaxPooling2D (2×2)	18×18×128	0
Flatten	41,472	0
Dense (512 units, ReLU, L2=0.01)	512	21,234,176
MCDropout (rate=0.5)	512	0
Dense (8 units, Softmax)	8	4,104

Total parameters: 21,330,952 (81.37 MB)

4.3 Key Design Choices

First Convolutional Layer — Linear Activation: In this case, a linear activation is applied to the first Conv2D layer instead of ReLU. This makes it possible for the network to extract both positive and negative feature mappings in the first layer before they get clipped, enabling the network to be flexible in extracting features.

L2 Regularization: Regularization of type L2 with weight 0.01 was done to Conv2D and Dense layers. It is aimed at penalizing the magnitude of weights during training, which means that the network will not remember the examples in the training data but rather learn how to generalize the facial features. This approach is especially important because of the Dense(512) layer, which contains 21 million parameters.

Monte Carlo Dropout (MCDropout): Unlike standard Dropout which is disabled during inference, MCDropout remains active during both training and prediction by always passing training=True to the dropout layer:


```
class MCDropout(keras.layers.Dropout):

    def call(self, inputs):

        return super().call(inputs, training=True)
```

This implies that with each pass, there will be some variance in output owing to randomly deactivating neurons. As a result, the network shows its uncertainty about faces that cannot be categorized by generating low confidence ratings rather than being too confident about an incorrect prediction. It will be helpful when considering the Unknown detection threshold since the network tends to generate low scores when faced with unknown images.

Glorot Uniform Initialization: Weights in the Dense layers are initialized using Glorot uniform (Xavier) distribution, where weight values depend on the number of inputs and outputs of the layer. This ensures that signal variance remains constant during the beginning of the learning process for improved convergence.

Softmax Output: The last layer employs Softmax activation, which transforms the logits to a probability distribution over the 8 classes. The class with the maximum probability is selected as the predicted output, while the probability itself serves as the confidence level in determining the Unknown class threshold.

4.4 Compilation

- **Loss function:** Categorical Crossentropy — standard choice for multi-class classification with Softmax output
- **Optimizer:** Adam with learning rate 0.0003 — adaptive learning rate optimizer that adjusts per-parameter learning rates, well suited for noisy gradients introduced by MCDropout
- **Metric:** Accuracy — proportion of correctly classified faces

5. Training Details

5.1 Training Configuration

The model was trained using the following configuration:

Parameter	Value
Optimizer	Adam
Learning rate	0.0003
Loss function	Categorical Crossentropy
Batch size	20

Parameter	Value
Max epochs	100

Early stopping patience 10

5.2 Callbacks

Two callbacks were used during training to ensure the best model was always preserved and unnecessary computation was avoided:

EarlyStopping: Monitors validation loss and stops training when no improvement is seen for 10 consecutive epochs. Automatically restores model weights from the best epoch when training stops. The patience value of 10 was chosen after experimentation — lower values caused training to stop too early before the model had fully converged.

ModelCheckpoint: Saves the model to disk (`face_recognition_model.keras`) whenever validation loss improves. This ensures the best model is always preserved on disk even if training continues beyond the optimal point.

5.3 Training Process

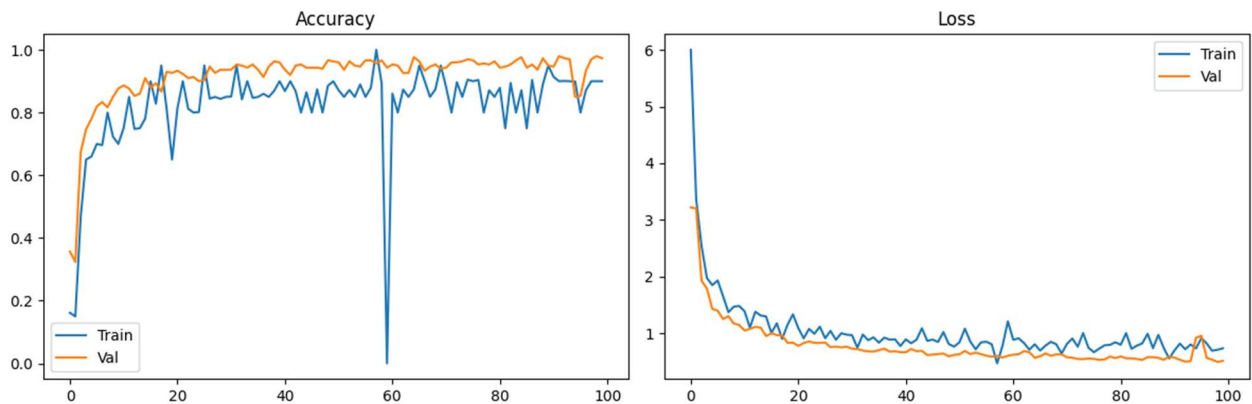
Training was done for up to 100 epochs using the augmented training generator with `steps_per_epoch=NUM_TRAINING//BATCH_SIZE`. Loss and accuracy during validation were tracked every epoch using the testing generator.

An interesting feature of the training graph is the presence of per-epoch accuracy variability due to MCDropout still being applied during inference. Given that neurons are randomly selected to be dropped on every forward pass, the accuracy at the batch level varies from one epoch to another. Nevertheless, the loss function curve, which averages out over multiple predictions, indicates smooth and steady convergence, proving that the neural network learns effectively even if the accuracy curve seems noisy. The optimal weight configuration corresponds to epoch 99 with validation loss of 0.493 and accuracy of 98%.

5.4 Training Curves

The training curves below show both accuracy and loss for the training and validation sets across all epochs:

- **Accuracy:** Both training and validation accuracy trend upward consistently, reaching 90–98% by the final epochs. The zigzag pattern in accuracy is expected due to MCDropout variance and does not indicate instability.
- **Loss:** Both training and validation loss decrease smoothly and converge in parallel, indicating the model is generalizing well without overfitting. The close tracking of train and validation loss throughout training confirms that L2 regularization and MCDropout are effectively preventing overfitting despite the model having 21 million parameters.



5.5 Label Encoding

Class labels (person names) are encoded as integers during training using a sorted list saved to `label_encoder.pickle`. This file maps class indices back to person names during inference in `final_recognition.py`, allowing the model's integer output to be converted to a human-readable name for display.

6. Evaluation Results

6.1 Classification Report

The model was evaluated on the 316-image held-out test set. The classification report below shows precision, recall, and F1-score for each individual:

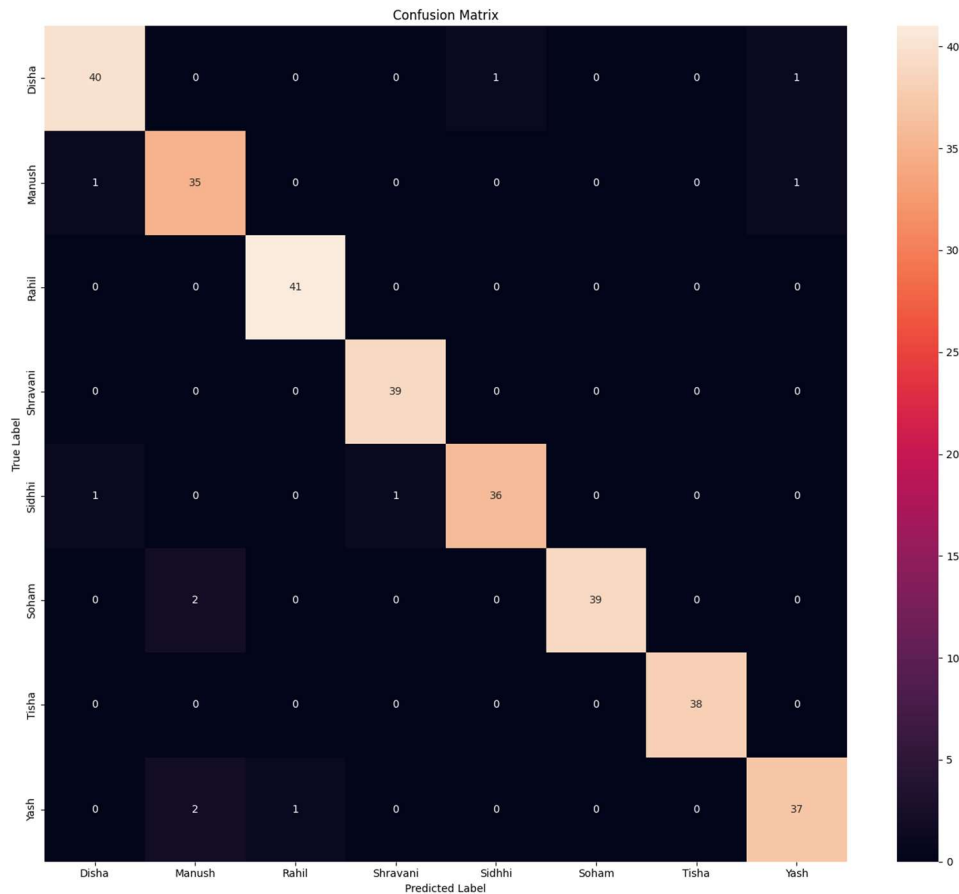
	precision	recall	f1-score	support
Disha	0.95	0.95	0.95	42
Manush	0.90	0.95	0.92	37
Rahil	0.98	1.00	0.99	41
Shravani	0.97	1.00	0.99	39
Sidhhi	0.97	0.95	0.96	38
Soham	1.00	0.95	0.97	41
Tisha	1.00	1.00	1.00	38
Yash	0.95	0.93	0.94	40
accuracy			0.97	316
macro avg	0.97	0.97	0.97	316
weighted avg	0.97	0.97	0.97	316

6.2 Metric Definitions

- **Precision:** Of all predictions made for a person, what proportion were correct.
- **Recall:** Of all actual images of a person, what proportion were correctly identified.

- **F1-Score:** The harmonic mean of precision and recall, providing a single balanced metric. Useful when both false positives and false negatives matter.
- **Support:** The number of test images for each person.

6.3 Confusion Matrix Analysis



The confusion matrix shows that the vast majority of predictions fall on the diagonal, indicating correct classification. The only notable confusions are:

- **Manush and Soham** — 2 misclassifications in each direction. Both are Indian males with similar beard patterns and facial structure, making them the most visually similar pair in the dataset.
- **Yash misclassified as Manush** — 2 errors, likely due to similar facial proportions.
- **Disha confused with Sidhhi and Yash** — 3 total errors, both females with similar facial geometry.

These confusions are expected and reflect genuine visual similarity between certain individuals rather than model failure. The fact that all confusions occur between visually similar people confirms the model is learning real facial features rather than superficial artifacts.

7. Threshold Selection

The empirical threshold value of 0.7 (70%) was derived from real-time webcam test results. The output of the CNN represents a probability vector for each of the 8 possible classes, calculated using the Softmax function. The maximum probability of the vector is selected as the confidence value for the classification decision. Without setting a threshold value, the network would always provide an identity among the 8 known identities even for an unknown person because of the property of probabilities summing up to 1.

During the testing phase, it was noticed that for a known individual, the network consistently returned high confidence results, more than 75-85%, whereas for an unknown individual, the probability scores varied unpredictably from 30-65%. There were no dominant probabilities for an unknown individual. The choice of threshold as 0.7 is ideal because it falls in between the above two intervals. Below 0.6, the network tends to classify unknown people into the group of known individuals, and above 0.8, there are cases when the network rejects known individuals due to difficult conditions like poor lighting and camera angles.

8. System Implementation

8.1 Overview

The real-time face recognition application runs as a desktop OpenCV window split into two panels — a live webcam feed on the left and an information sidebar on the right. It uses two models in sequence: MTCNN handles face detection and the custom CNN handles identity classification. Recognized individuals are logged automatically to an attendance file.

8.2 Processing Pipeline

For every frame captured from the webcam:

1. The frame is resized to 600px width and converted to RGB
2. MTCNN detects all faces in the frame and returns their bounding box coordinates
3. Each detected face is cropped and clamped to frame boundaries
4. The face is converted to grayscale, resized to 160×160, normalized to [0,1], and reshaped to (1, 160, 160, 1) for CNN input
5. The CNN outputs a Softmax probability distribution across 8 classes
6. If the highest probability exceeds 0.7, the person is identified by name. Below 0.7 the face is labeled "Unknown"
7. A bounding box and label are drawn on the frame
8. Recognized individuals are written to attendance.csv

8.3 User Interface

The display window has two panels side by side:

Left panel — live webcam feed:

- Bounding box around each detected face — green for a recognized person, red for Unknown

- Name label displayed above the bounding box
- Confidence score shown as a percentage next to the name

Right panel — information sidebar:

- Application title at the top
- Currently detected person's name, colored green if recognized and red if Unknown or no face detected
- Confidence percentage and a visual bar that fills proportionally to the confidence score
- Live clock showing the current date and time
- "Press Q to quit" reminder at the bottom

9. Limitations

Fixed identity set: With the current approach, retraining of the model is needed each time a new individual needs to be included in the recognition system. This poses challenges in practical applications because it would be unrealistic to train the system from scratch each time a new user needs to be added.

Lighting dependency: Although brightness changes are involved in data augmentation, the model performs optimally in lighting environments close to what was used during recording. Extreme cases like extremely dark rooms, strong backlighting, or coloured lights significantly impact the performance negatively.

Single face performance: The model is trained and evaluated on single-face images mostly. In cases where there are two or more faces in one image, the detection process still functions correctly but could be less accurate, especially when faces are near each other or overlap.

MCDropout variance: Because MCDropout remains active during inference, predictions vary slightly between frames for the same person. This can cause the displayed name to flicker between a person's name and "Unknown" even when the face is clearly visible and well-lit, which is visually distracting.

Dataset size: While 186–208 images per person produced strong results for our 8 subjects, the model may struggle to generalize to more people or to the same people under significantly different conditions such as new hairstyles, glasses, or facial hair changes.

10. Future Improvements

Siamese Network: With the softmax classifier being replaced with a Siamese CNN to learn similarity among the pairs of faces, it would be possible for us to enrol new individuals using only a handful of reference faces without requiring any form of re-training. However, we experimented with this strategy throughout the course of our project and developed a functional prototype, but the present CNN model still performed better.

Transfer Learning: Utilizing a pretrained network like FaceNet or ArcFace, which have been trained on millions of face images, and performing fine-tuning of the last classifier layer with our

own dataset will probably help us achieve over 99% accuracy with fewer samples and reduced time consumption.

Liveness Detection: The liveness detection model (liveness.h5) was trained separately for this project to distinguish between real faces and fake faces based on a CNN that was trained on real faces and spoofed faces. Incorporating this in the overall system architecture would help in preventing photo and video spoofing attacks.

Temporal Smoothing: Averaging CNN predictions over several consecutive frames would reduce the flickering caused by MCDropout variance and produce more stable name labels in the UI.

Web Interface: Replacing the OpenCV desktop window with a Flask web application would allow the system to be accessed from any device on the same network, making it more practical for shared environments like office entrances or classrooms.

Larger and More Diverse Dataset: Capturing video footage of each individual in various settings, lighting situations, and time spans would increase the reliability of the model. Capturing video footage of individuals wearing and not wearing glasses, having varying haircuts, and wearing different attire according to season would enhance generalizability.

11. Conclusion and References

The aim of this project was to design an online face recognition application based on a CNN model trained from scratch using self-generated datasets. We reached an accuracy rate of 97% using the final model across 8 individuals where Rahil, Shravani, and Tisha have 100% precision while none has less than 92% F1 score.

The most surprising fact about our experience is that none of our model designs was responsible for this efficiency. The first model design, when trained on dim videos alongside images downloaded from the internet, had 22% accuracy while recording good-quality videos led us to 97% accuracy without changing our model design.

Our architectural experiments had another lesson to offer. At first, the CNN was too thin and degenerated into assigning a single class for everybody. The addition of another convolutional layer, boosting the number of neurons in the dense layer up to 512 and reducing the learning rate to 0.0003 was enough to bring the model above 60%. However, although MCDropout introduced noticeable accuracy variations in the training process, it was useful for inference when an unknown face yielded a low scatter in confidence measures, thus allowing us to use 0.7 as a threshold value.

The remaining confusions — Manush/Soham, Yash/Manush — occur between the most visually similar individuals in the dataset and are expected. They reflect the genuine limits of a model trained on 186–208 images per person rather than millions.

The system works. It runs in real time, recognizes known faces reliably, rejects unknown ones, and logs attendance automatically. The obvious next step is replacing the fixed classifier with a Siamese network so new people can be enrolled without retraining — something we prototyped during this project but did not have enough training pairs to make competitive.

References

1. Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*. Available at: <https://opencv.org>
2. Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks. *IEEE Signal Processing Letters*, 23(10), 1499–1503. (MTCNN)
3. Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Available at: <https://www.tensorflow.org>
4. Chollet, F., et al. (2015). Keras. Available at: <https://keras.io>
5. Jiang, H., & Learned-Miller, E. (2017). Face Detection with the Faster R-CNN. In *Proceedings of the IEEE International Conference on Automatic Face & Gesture Recognition*. (SSD face detector — res10_300x300_ssd_iter_140000)
6. Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning. *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. (Monte Carlo Dropout)
7. Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*. (Glorot Uniform Initialization)
8. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. (Adam Optimizer)